

OBJECT ORIENTED PROGRAMMING WITH JAVA — MARKING GUIDE

Academic Year: 2025–2026

Level: Year 2 (A, B, C)

Term: I

Total Marks: 100

Time: 180 minutes

SECTION A — Multiple Choice (Each 2.5 Marks × 20 = 50 Marks)

Qn	Correct Answer	Marks (2.5)
1	Encapsulation	2.5
2	Calls another constructor in the same class	2.5
3	FileWriter	2.5
4	Compilation error	2.5
5	Executes always	2.5
6	FileInputStream	2.5
7	abstract void method();	2.5
8	Any class that is a subclass of Number	2.5
9	Interface	2.5
10	Character	2.5
11	IOException	2.5
12	Current number of elements	2.5
13	Derived display	2.5
14	No	2.5
15	JVM terminates program	2.5

16	A placeholder for a specific type	2.5
17	2	2.5
18	Type safety and code reusability	2.5
19	Writes "Hello, world!" to a file named 'test.txt'	2.5
20	extends	2.5

SECTION B — Short Answer (Any 4 × 5 = 20 Marks)

- Compile-time vs Runtime Polymorphism: Compile-time: method overloading (checked by compiler). Runtime: method overriding (resolved by JVM). Include examples. (5 Marks)
- WORA Principle: Write Once, Run Anywhere; Java bytecode runs on any JVM platform. (5 Marks)
- Collection vs Collections: Collection is an interface; Collections is a utility class with static methods like sort(), reverse(). (5 Marks)
- Multiple Inheritance in Java: Achieved through interfaces. Use implements keyword to demonstrate. (5 Marks)
- Encapsulation Importance: Encapsulation hides data and provides controlled access via getters/setters for software security. (5 Marks)

SECTION C — Practical Coding (Mandatory, 2 Questions = 30 Marks)

Question 21 — Generic Class Box<T> (15 Marks)

```
class Box<T> {
    private T value;
    public void setValue(T value) { this.value = value; }
    public T getValue() { return value; }
}
```

```
public class Main {
    public static void main(String[] args) {
        Box<Integer> intBox = new Box<>();
        Box<String> strBox = new Box<>();
        intBox.setValue(100);
        strBox.setValue("Java");
        System.out.println(intBox.getValue());
        System.out.println(strBox.getValue());
    }
}
```

```
}  
}
```

Marking Breakdown:

- Generic class definition with <T> → 3 marks
- Private field and setter/getter → 4 marks
- Proper main method creation → 3 marks
- Object instantiation and value assignment → 3 marks
- Correct output (display values) → 2 marks

Total: 15 marks

Question 22 — Student Class with Comparable and Comparator (15 Marks)

```
import java.util.*;
```

```
class Student implements Comparable<Student> {  
    int id;  
    String firstName;  
    String lastName;  
    double grade;  
  
    public Student(int id, String firstName, String lastName, double grade) {  
        this.id = id; this.firstName = firstName; this.lastName = lastName; this.grade = grade;  
    }  
  
    public int compareTo(Student s) {  
        return Integer.compare(this.id, s.id);  
    }  
  
    @Override  
    public boolean equals(Object o) {  
        if (this == o) return true;  
        if (!(o instanceof Student)) return false;  
        Student s = (Student) o;  
        return id == s.id && lastName.equals(s.lastName);  
    }  
  
    @Override
```

```

    public int hashCode() {
        return Objects.hash(id, lastName);
    }

    @Override
    public String toString() {
        return id + " " + firstName + " " + lastName + " " + grade;
    }
}

class GradeComparator implements Comparator<Student> {
    public int compare(Student s1, Student s2) {
        return Double.compare(s2.grade, s1.grade); // descending
    }
}

public class Main {
    public static void main(String[] args) {
        List<Student> list = new ArrayList<>();
        list.add(new Student(2, "Alice", "Smith", 85.5));
        list.add(new Student(1, "Bob", "Jones", 90.0));
        list.add(new Student(3, "Charlie", "Brown", 80.0));

        Collections.sort(list);
        System.out.println("Sorted by ID:");
        list.forEach(System.out::println);

        Collections.sort(list, new GradeComparator());
        System.out.println("\nSorted by Grade:");
        list.forEach(System.out::println);
    }
}

```

Marking Breakdown:

- Attributes + constructor → 3 marks
- Implements Comparable by id → 3 marks
- Comparator for grade descending → 3 marks
- Proper equals() and hashCode() → 3 marks
- List creation and sorting (both methods) → 3 marks

Total: 15 marks

✓ Grand Total: 100 Marks